

PulseHR Repository Report

Completed implementation and verification report

Human Resource Management System for Indian SMEs

This report explains the completed frontend API integration, payroll correction, security controls, testing and GitHub delivery. SSO and full field-level encryption are intentionally outside the required scope for this submission.

GitHub: github.com/V44M4N/PulseHR | Branch: `codex/pulsehr-integration` | Latest commit: `117f581`

Executive summary

PulseHR began as two substantial but disconnected applications: a React/Vite frontend backed entirely by mock data and a Node/Express/Prisma backend with real business logic. The completed implementation joins these halves through a reusable API client and TanStack Query data layer. Login is real, roles come from the backend, and the major HR screens now read live records through their matching endpoints.

The payroll placeholder has been removed. Payroll now reads an employee-specific stored salary structure, while PF, TDS, HRA and net-pay calculations remain explicit and tested. The application also includes TOTP MFA, administrator IP allowlisting, secure token refresh and masking for sensitive PAN and bank account values.

Current release position: the frontend and backend are connected across the planned modules, the security controls required for this submission are implemented, the builds and tests pass, and the current branch is pushed to GitHub.

Repository at a glance

Area	Current state	Evidence
Frontend	React 18, TypeScript, Vite and TanStack Query connected to real backend APIs across the HR modules.	Frontend build passes; 15 tests pass.
Backend	Express, Prisma, PostgreSQL, Redis. Auth, leave and payroll flows execute against the database.	Backend build passes; 7 tests pass.
Database	Prisma schema includes a new SalaryStructure model linked one-to-one to Employee.	Prisma generation and Docker startup pass.
Delivery	Completed code and the updated report are committed and pushed to the private PulseHR GitHub repository.	Branch is tracking origin.

Implemented changes

Authentication

The frontend now has a real login form at /login. Access tokens stay in module memory, refresh tokens use the backend HttpOnly cookie, and the API client attaches bearer authorization. A 401 triggers one shared refresh operation for concurrent requests and retries the original request once. A failed refresh or second 401 clears the session and React Query cache. The previous demo role switcher is gone; navigation reflects the authenticated role returned by the backend. The login form also accepts an authenticator code when MFA is enabled.

Frontend module rollout

Attendance now reads today, weekly and monthly records and sends clock-in and clock-out actions. Payslips, documents, expenses, helpdesk, feed, employees, recruitment, onboarding, payroll, analytics and admin screens now query their backend modules. The dashboard combines live leave, attendance and feed data. The pages keep their existing visual structure and now show loading and retryable error states.

Payroll salary source

SalaryStructure stores monthly gross pay and calculation parameters per employee. Payroll processing fails clearly when a structure is missing instead of inventing a salary from an internal ID. The pure calculator supports custom basic, HRA, PF, TDS and professional-tax values.

Reusable module layer

The frontend includes an endpoint map and generic query and mutation helpers for the connected modules. This gives the remaining product work a consistent pattern for cache keys, request errors, loading states and post-mutation refreshes.

Verification results

Check	Result	Scope
Frontend production build	PASS	Vite bundle generated successfully.
Frontend type-check	PASS	tsc --noEmit -p tsconfig.app.json.
Frontend tests	PASS - 15 tests	API transport, refresh concurrency, timeout, role restoration, logout cache clearing.
Backend production build	PASS	TypeScript compilation and Prisma client generation.
Backend tests	PASS - 7 tests	Refresh cookie transport and salary calculations.
Auth Docker smoke test	PASS - all four roles	Login, identity, role enforcement, refresh rotation, replay rejection and logout revocation.
Leave Docker smoke test	PASS - employee flow	Create, read, cancel and balance restoration.
Payroll Docker smoke test	PASS - 8 seeded employees	Process run with stored salary structures and aggregate totals.
Browser verification	PASS - session lifecycle	Protected redirect, real sign-in, reload restoration, sign-out and signed-out reload.
Security controls	PASS	TOTP MFA endpoints and login enforcement, administrator IP allowlist middleware and PII masking compile successfully.
Git delivery	PASS	Changes committed and pushed to github.com/V44M4N/PulseHR .

Security and data handling

The access token is not persisted in `localStorage` or `sessionStorage`. Refresh rotation is enforced by the backend session table, and logout blocklists the access token in Redis. Existing Helmet, rate limiting, audit middleware, Zod validation and role middleware remain active. TOTP MFA can be enrolled and verified, administrator routes enforce the configured IP allowlist, and PAN and bank account values are masked in employee API responses.

Scope decision

SSO and full field-level encryption are not required for this submission. SSO needs a selected identity provider and credentials. Full encryption needs a key-management plan, a migration and controlled re-encryption of existing records. The current implementation includes the requested practical protections for the local demonstration and mentor review.

How to explain the work

I connected the frontend screens to the backend server using one common API client. Login is now real, and the system knows the user role from the database. Each major HR screen gets its data from the correct backend endpoint instead of the old hardcoded demo file. I added loading and error messages so users know what is happening.

I also added authenticator-code MFA, IP restrictions for administrators, secure token refresh and masking of sensitive PAN and bank details. Payroll now uses each employee salary structure stored in the database. Finally, I built and tested both applications and pushed the working code to GitHub.

Remaining roadmap

Workstream	Status and next action
Phase 3 UI rollout	Completed for the requested module connection scope. Future improvements can add richer create and edit forms to screens that currently focus on live reads.
Phase 5 coverage	Baseline auth and payroll coverage is complete. More integration cases can be added for protected resource ownership and month-boundary edge cases.
Phase 6 security	Required scope completed with TOTP MFA, IP allowlist enforcement and PII masking. SSO and full field-level encryption are excluded by project decision.
Production hardening	Run migrations instead of db push, configure real secrets, verify HTTPS cookie behavior, set up observability, and perform a security review.

Demo accounts

The development seed uses Password@123 for admin@pulsehr.dev, hr@pulsehr.dev, manager@pulsehr.dev and emp1@pulsehr.dev. These credentials are for the disposable local environment only and must not be reused in any shared or production system.